

Architecture et Programmation Web

Séance 4 : Formulaires et Traitement des Entrées

Nicolas Burny

21 février 2026

ECAM

Introduction aux Formulaires

- Jusqu'à présent, nous avons surtout **affiché** des données.
- Pour une application réelle, l'utilisateur doit pouvoir **envoyer** des informations :
 - Créer un compte.
 - Publier un message.
 - Rechercher un produit.
 - Configurer des préférences.
- Le **formulaire HTML** est l'outil principal de cette interaction.

- **GET** : Utilisé pour demander une ressource.
 - Les données sont dans l'URL (?query=abc).
 - Limité en taille, visible dans l'historique.
 - Idéal pour les **recherches** et le **filtrage**.
- **POST** : Utilisé pour envoyer des données qui modifient l'état du serveur.
 - Les données sont dans le *body* de la requête.
 - Pas de limite de taille (fichiers), données non visibles dans l'URL.
 - Idéal pour la **création**, **modification** ou **connexion**.

Structure HTML d'un formulaire

```
<form action="/submit" method="POST">
  <div class="mb-3">
    <label for="title">Titre :</label>
    <input type="text" id="title" name="title" required>
  </div>
  <div class="mb-3">
    <label for="description">Description :</label>
    <textarea id="description" name="description"></textarea>
  </div>
  <button type="submit">Envoyer</button>
</form>
```

Important

L'attribut name est crucial : c'est la clé que Flask utilisera pour récupérer la valeur.

L'objet Request dans Flask

Flask fournit l'objet global `request` (à importer) pour extraire les informations envoyées par le client.

- `request.method` : Retourne "GET" ou "POST".
- `request.args` : Dictionnaire contenant les paramètres d'URL (GET).
- `request.form` : Dictionnaire contenant les données du corps (POST).
- `request.files` : Pour les téléchargements de fichiers.

Exemple de traitement POST

```
from flask import Flask, request, render_template

@app.route('/add', methods=['GET', 'POST'])
def add_feature():
    if request.method == 'POST':
        # Récupération via les 'name' HTML
        title = request.form.get('title')
        description = request.form.get('description')

        # Logique de traitement...
        return f"Ajout de {title} avec description {description}"

    return render_template('form.html')
```

Entrées complexes et Listes

Gérer les Checkboxes

- Contrairement aux champs texte, une checkbox n'est envoyée que si elle est cochée.
- Si plusieurs checkboxes partagent le même name, `request.form.get()` ne retournera que la dernière valeur.
- Solution : `request.form.getlist('nom')`.

```
# HTML: <input type="checkbox" name="tags" value="Bug">
```

```
# HTML: <input type="checkbox" name="tags" value="UI">
```

```
tags = request.form.getlist('tags')
```

```
# tags = ['Bug', 'UI']
```

Upload de Fichiers

1. **Côté HTML** : Ajouter l'attribut enctype.

```
<form method="POST" enctype="multipart/form-data">  
    <input type="file" name="attachment">  
</form>
```

2. **Côté Python** : Utiliser request.files.

```
file = request.files['attachment']  
if file:  
    file.save('static/uploads/' + file.filename)
```

Danger : Nom de fichier malveillant

Un utilisateur peut nommer son fichier `../../config.py`. Il ne faut **jamais** utiliser le nom original directement.

- Utiliser `secure_filename` de `werkzeug.utils`.
- Vérifier l'extension (ex : uniquement `.jpg`, `.png`).
- Limiter la taille maximale (`MAX_CONTENT_LENGTH`).

Validation et Sécurité

Règle d'or : Ne jamais faire confiance à l'utilisateur.

- Même si vous avez des attributs `required` en HTML, un utilisateur malveillant peut les contourner via la console ou *Postman*.
- Vous devez **re-valider** les données en Python :
 - Vérifier que le champ n'est pas vide.
 - Vérifier la longueur minimale/maximale.
 - Vérifier le type (nombre, email).

Exemple de validation simple

```
if request.method == 'POST':  
    title = request.form.get('title')  
  
    if not title or len(title) < 3:  
        error = "Le titre est trop court !"  
        return render_template('form.html', error=error)  
  
    # Si OK, suite du traitement...
```

Feedback : Sessions et Flash Messages

- Comment informer l'utilisateur que son action a réussi après une redirection ?
- Le protocole HTTP est "sans état" (*stateless*).
- Flask propose le système de **Flash Messages** :
 1. On stocke un message temporaire en session.
 2. On redirige l'utilisateur.
 3. Le template affiche le message puis le supprime automatiquement.

Utiliser flash()

Côté Python :

```
from flask import flash, redirect, url_for

@app.route('/submit', methods=['POST'])
def submit():
    flash("Données enregistrées !", "success")
    return redirect(url_for('index'))
```

Côté Template (base.html) :

```
{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert alert-{{ category }}">{{ message }}</div>
    {% endfor %}
  {% endif %}
```

Le pattern Post-Redirect-Get (PRG)

- Si vous renvoyez du HTML directement après un POST (`render_template`)...
- ... et que l'utilisateur rafraîchit la page (F5)...
- ... le navigateur propose de **renvoyer les données**.
- **Risque** : Doublons en base de données (ex : deux paiements, deux messages).

Le pattern **PRG** consiste à :

1. Recevoir le **POST**.
2. Effectuer le traitement.
3. Répondre par une **Redirection** (302) vers une page en **GET**.
4. Le navigateur demande alors la nouvelle page en **GET**.

Bénéfice

Un rafraîchissement de page ne fera que redemander la page en GET, sans renvoyer le formulaire.

Questions ?